



University of Science, VNU - HCM

Consistency Models for purifying poisoning

Name: **Le Tien Dat**, Student ID: **23125028**

Name: **Phan Tuan Kiet**, Student ID: **23125062**

Class: **23APCS2**

May 26, 2026.

Contents

1	Threat Model: Targeted Poisoning Attack	2
1.1	Gradient Matching	3
1.2	Feature Space Centering (Bullseye Polytope)	4
2	Proposed Methodology: Pixel-Space Consistency Purification	5
2.1	Why Consistency Models over Diffusion Models?	5
2.2	Universal Offline Poison Generation	6
2.3	Manifest Allocator for CIFAR-10	7
2.4	Forward Process	8
2.5	Training Objectives	9
3	Algorithms and Implementation Details	10
3.1	Offline Dataset Generation	10
3.2	Training Process	11
3.3	Inference Process	13
3.4	Pre-generated Evaluation Cases from the Training Split	14
3.5	Metrics	14

Abstract

This proposal presents a fast poison-aware purification framework based on a one-step pixel-space Consistency Model (CM). The goal is to sanitize poisoned training images before downstream model training while preserving the semantic content and clean labels of the original data. Diffusion-based purification is a strong generative defense because it can progressively remove small adversarial perturbations and recover natural-looking images. However, its major limitation is inference time: standard diffusion purification requires many iterative denoising steps, making it expensive for dataset-level sanitization. This becomes impractical when every image in a training set must be purified.

To address this problem, we propose training a pixel-space CM that learns a direct mapping from a poison-aware noised image to its clean counterpart. After training, the purifier can sanitize images using a single neural function evaluation, greatly reducing the cost compared with iterative diffusion models. We construct an offline paired dataset containing Witches’ Brew poisons, Bullseye Polytope poisons, and clean identity pairs. This allows the model to learn a general poison-removal behavior instead of overfitting to a single attack pattern. The resulting framework is designed to be fast, batchable, and suitable for CIFAR-10 scale experiments.

1 Threat Model: Targeted Poisoning Attack

The attack considered in our experiments is the Targeted Data Poisoning attack derived from the “Witches’ Brew: Industrial Scale Data Poisoning via Gradient Matching” [1] and “Bullseye Polytope: A Scalable Clean-Label Poisoning Attack with Improved Transferability” [2] papers. To formally define our threat landscape, we specify the following components:

- **Attacker’s Objective:** The adversary aims to cause a targeted misclassification. Specifically, they want a victim model trained on the poisoned dataset to misclassify a particular, unmodified target image at test time, without noticeably degrading the model’s overall accuracy on clean data.
- **Attacker’s Capabilities:** The attacker has the ability to inject a small fraction of perturbed training data (budget of 1.0%) into the training set. The attack is strictly “clean-label,” meaning the attacker cannot alter the semantic labels of the injected images, and the perturbations are strictly bounded (e.g., small ℓ_∞ norm constraint) making them nearly imperceptible.

- **Attacker’s Knowledge:** We assume a **Gray-Box** setting for the poison generation. The attacker has knowledge of the target model’s architecture to calculate gradients during the poison crafting phase, but has no knowledge of the specific training procedure (e.g., random initialization, mini-batching, or data augmentation) used by the victim.
- **Defender’s Capabilities:** The defender controls the training pipeline and trains the victim model (e.g., ResNet-18) “from scratch” on the provided dataset. Crucially, the defender lacks prior knowledge of whether the dataset is poisoned or what specific attack methodology is employed. Thus, the defender employs attack-agnostic pre-processing or purification mechanisms (such as diffusion models or frequency detectors) to blindly sanitize the data without relying on a specific threat signature.

1.1 Gradient Matching

Data poisoning is fundamentally a bilevel optimization problem. Because the victim model is trained via first-order optimization (gradient descent), the attacker can manipulate the final model by ensuring the training data produces a malicious gradient signal.

The core idea of the Witches’ Brew attack is **Gradient Matching** [1]. If the attacker can constrain the gradients of the poisoned training data to mimic the gradient of the target image evaluated with the adversarial label, the victim will unwittingly minimize the adversarial loss during standard training. Unconstrained, the optimal condition is:

$$\nabla_{\theta} \mathcal{L}(F(x^t, \theta), y^{\text{adv}}) \approx \frac{1}{P} \sum_{i=1}^P \nabla_{\theta} \mathcal{L}(F(x_i + \Delta_i, \theta), y_i) \quad (1)$$

where (x^t, y^{adv}) is the target image and its malicious label, P is the number of poisoned images, and Δ_i is the imperceptible perturbation added to the clean training image x_i (bounded by $\|\Delta_i\|_{\infty} \leq \varepsilon$).

Because gradient magnitudes fluctuate drastically during a full training run from scratch, attempting to match exact gradient values across all training stages is infeasible. Instead, Witches’ Brew achieves high transferability to newly initialized models by maximizing the *co-sine similarity* (aligning the directions) between the target gradient and the poisoned data gradients on a static, pre-trained model parameter vector θ . The attacker minimizes the following alignment loss $\mathcal{B}(\Delta, \theta)$:

$$\mathcal{B}(\Delta, \theta) = 1 - \frac{\langle \nabla_{\theta} \mathcal{L}(F(x^t, \theta), y^{\text{adv}}), \sum_{i=1}^P \nabla_{\theta} \mathcal{L}(F(x_i + \Delta_i, \theta), y_i) \rangle}{\|\nabla_{\theta} \mathcal{L}(F(x^t, \theta), y^{\text{adv}})\| \cdot \|\sum_{i=1}^P \nabla_{\theta} \mathcal{L}(F(x_i + \Delta_i, \theta), y_i)\|} \quad (2)$$

To ensure these crafted poisons survive standard deep learning pipelines, the attack utilizes **Differentiable Data Augmentation**. By applying random crops, translations, and flips to the poisoned images during the optimization of Equation (2) and differentiating through these operations, the generated Δ_i becomes highly robust, allowing the attack to successfully breach models trained entirely from scratch.

1.2 Feature Space Centering (Bullseye Polytope)

While Witches’ Brew targets from-scratch training via gradient matching, the Bullseye Polytope (BP) attack [2] is a clean-label method heavily optimized for transfer learning. Instead of manipulating gradients, BP directly manipulates the victim’s *feature space*. It builds upon the Convex Polytope attack, which attempts to surround a target image’s feature representation with poison samples, tricking the downstream classifier into assigning the poison label to the target.

However, merely placing the target inside the boundary of the poison convex hull leaves the attack vulnerable to feature space shifts during model fine-tuning. BP solves this by forcing the target directly to the *center* (mean embedding) of the poison samples. Given m substitute feature extractors $\phi^{(i)}$, a target image x_t , and k clean base images $x_b^{(j)}$, the attacker crafts poisons $x_p^{(j)}$ by minimizing:

$$\min_{\{x_p^{(j)}\}} \frac{1}{2m} \sum_{i=1}^m \frac{\left\| \phi^{(i)}(x_t) - \frac{1}{k} \sum_{j=1}^k \phi^{(i)}(x_p^{(j)}) \right\|^2}{\left\| \phi^{(i)}(x_t) \right\|^2} \quad (3)$$

subject to the imperceptibility bound $\|x_p^{(j)} - x_b^{(j)}\|_\infty \leq \varepsilon$.

By predetermining the convex combination coefficients as uniform weights ($\frac{1}{k}$), BP eliminates a highly expensive coefficient-optimization step required by prior methods. This makes poison generation an order of magnitude faster. Furthermore, by centering the target deep within the attack zone—the “bullseye”—the attack maximizes its distance from the polytope boundaries. This drastically improves attack transferability to black-box models and ensures the poison survives even in end-to-end transfer learning scenarios where the feature extractor itself is altered.

2 Proposed Methodology: Pixel-Space Consistency Purification

To neutralize targeted clean-label poisoning attacks efficiently, we propose a **One-Step Pixel-Space Consistency Purifier**, which derived from the distillation process of [3] and [4]. The purifier is trained to map a noised and potentially poisoned image back to its corresponding clean image. The central motivation is speed: instead of running a long reverse diffusion chain, the model learns to directly recover the clean sample in one step.

Let $\mathbf{x}_{\text{untrusted}}$ be an image from an untrusted training set. The purifier produces

$$\hat{\mathbf{x}} = P_{\theta}(\mathbf{x}_{\text{untrusted}}), \quad (4)$$

where $\hat{\mathbf{x}}$ should preserve the semantic label of the image while removing hidden poison perturbations.

The proposed pipeline has three main components:

- an offline poison-pair dataset containing clean and poisoned image pairs;
- a poison-aware forward noising process that models poisoning perturbations as structured noise;
- a one-step consistency purifier trained to recover clean images from the poison-aware noised inputs.

2.1 Why Consistency Models over Diffusion Models?

Diffusion models can purify adversarially perturbed images by adding noise and then denoising the image back toward the natural data distribution. This is effective, but the reverse process usually requires many neural network evaluations. If a defender wants to sanitize an entire training dataset, this cost becomes very large.

For example, a diffusion purifier may require tens or hundreds of denoising steps for each image:

$$\mathbf{x}_t \rightarrow \mathbf{x}_{t-1} \rightarrow \cdots \rightarrow \mathbf{x}_0. \quad (5)$$

Thus, the total cost scales with both the dataset size and the number of denoising steps.

A Consistency Model is more suitable for our goal because it learns a direct mapping:

$$f_{\theta}(\mathbf{x}_t, t) \approx \mathbf{x}_0. \quad (6)$$

Instead of repeatedly denoising the image step by step, the CM predicts the clean image directly from a noised state. Therefore, after training, purification can be performed with one neural function evaluation:

$$\hat{\mathbf{x}}_0 = f_{\theta}(\mathbf{x}_{t^*}, t^*). \quad (7)$$

This significantly improves inference speed and makes dataset-level purification more practical. The main benefit of the proposed approach is therefore:

$$\text{Diffusion purification: many denoising steps} \longrightarrow \text{CM purification: one denoising step.} \quad (8)$$

2.2 Universal Offline Poison Generation

We construct an offline paired dataset for purifier training. Let

$$\mathcal{D}_{\text{clean}} = \{(\mathbf{x}_{\text{clean}}^{(i)}, y^{(i)})\}_{i=1}^N \quad (9)$$

be the clean CIFAR-10 training dataset. We generate a paired dataset

$$\mathcal{D}_{\text{pair}} = \left\{ \left(\mathbf{x}_{\text{clean}}^{(i)}, \mathbf{x}_{\text{poison}}^{(i)}, y^{(i)}, m^{(i)} \right) \right\}_{i=1}^M, \quad (10)$$

where $\mathbf{x}_{\text{clean}}^{(i)}$ is the original clean image, $\mathbf{x}_{\text{poison}}^{(i)}$ is its poisoned counterpart, $y^{(i)}$ is the original clean label, and $m^{(i)}$ stores metadata such as attack type, poison class, target class, random seed, and perturbation bound.

The poison residual is

$$\delta^{(i)} = \mathbf{x}_{\text{poison}}^{(i)} - \mathbf{x}_{\text{clean}}^{(i)}. \quad (11)$$

The paired dataset contains three types of examples:

- Witches' Brew poison-clean pairs;
- Bullseye Polytope poison-clean pairs;
- clean identity pairs, where $\mathbf{x}_{\text{poison}} = \mathbf{x}_{\text{clean}}$.

The clean identity pairs are important because the purifier will be applied blindly to all images at inference time. Therefore, it should learn not to unnecessarily modify clean samples.

The target training dataset size is:

10000 Witches’ Brew pairs + 10000 Bullseye Polytope pairs + 10000 clean identity pairs = 30000 pairs.

This dataset is not intended to represent one single poisoned victim dataset. Instead, it is a controlled poison bank for training a general purifier.

2.3 Manifest Allocator for CIFAR-10

To prevent accidental data leakage, we use a deterministic manifest allocator. CIFAR-10 contains 5000 training images per class. For each class c , we first shuffle all image indices of that class using a fixed random seed. Then we allocate non-overlapping blocks:

$[0, 499] \rightarrow$ Witches’ Brew training case 1,
 $[500, 999] \rightarrow$ Witches’ Brew training case 2,
 $[1000, 1499] \rightarrow$ Witches’ Brew evaluation case,
 $[1500, 2499] \rightarrow$ Bullseye Polytope training pool,
 $[2500, 2519] \rightarrow$ Bullseye Polytope evaluation pool,
 $[2520, 3519] \rightarrow$ clean identity training pool,
 $[3520, 4999] \rightarrow$ reserve and target-selection pool.

This split ensures that the clean base images used for Witches’ Brew training, Bullseye Polytope training, clean identity training, and evaluation are disjoint.

For Witches’ Brew, each class contributes two training cases:

$$10 \times 2 \times 500 = 10000$$

poison-clean pairs.

For Bullseye Polytope, each class contributes 100 groups of 10 images:

$$10 \times 100 \times 10 = 10000$$

poison-clean pairs.

For clean identity training, each class contributes 1000 clean images:

$$10 \times 1000 = 10000$$

clean pairs.

2.4 Forward Process

A standard Consistency Model satisfies the boundary condition:

$$f_\theta(\mathbf{x}, 0) = \mathbf{x}. \quad (12)$$

If we directly use $\mathbf{x}_{\text{poison}}$ as the root of the noising process, then near $t = 0$, the model is encouraged to reconstruct $\mathbf{x}_{\text{poison}}$. However, the desired output is $\mathbf{x}_{\text{clean}}$. This creates an undesirable conflict.

To avoid this issue, we use $\mathbf{x}_{\text{clean}}$ as the root and treat the poison residual as a structured noise component. Let

$$\delta = \mathbf{x}_{\text{poison}} - \mathbf{x}_{\text{clean}}. \quad (13)$$

The poison-aware forward process is:

$$\mathbf{x}_t^* = \sqrt{\bar{\alpha}_t} \mathbf{x}_{\text{clean}} + \sqrt{1 - \bar{\alpha}_t} (\epsilon + \gamma_a \delta), \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad (14)$$

where $a \in \{\text{WB}, \text{BP}, \text{clean}\}$ denotes the example type and γ_a controls the strength of the poison residual.

As $t \rightarrow 0$,

$$\sqrt{1 - \bar{\alpha}_t} \rightarrow 0, \quad (15)$$

so both the Gaussian noise and the poison residual vanish:

$$\mathbf{x}_t^* \rightarrow \mathbf{x}_{\text{clean}}. \quad (16)$$

Thus, the boundary condition is naturally satisfied:

$$f_\theta(\mathbf{x}_{\text{clean}}, 0) = \mathbf{x}_{\text{clean}}. \quad (17)$$

The model is trained to predict:

$$\hat{\mathbf{x}}_0 = f_\theta(\mathbf{x}_t^*, t) \approx \mathbf{x}_{\text{clean}}. \quad (18)$$

2.5 Training Objectives

The student model predicts:

$$\hat{\mathbf{x}}_0 = f_\theta(\mathbf{x}_{t_{n+1}}^*, t_{n+1}). \quad (19)$$

1. Consistency Distillation Loss. This loss encourages the student and teacher to map different points on the same trajectory to the same clean origin. Let $\hat{\mathbf{x}}_{t_n}^\Psi$ be a shallower timestep estimate obtained by an ODE solver Ψ . Then:

$$\mathcal{L}_{\text{distill}} = d\left(f_\theta(\mathbf{x}_{t_{n+1}}^*, t_{n+1}), \text{sg}\left[f_{\theta-}(\hat{\mathbf{x}}_{t_n}^\Psi, t_n)\right]\right), \quad (20)$$

where $\text{sg}[\cdot]$ stops gradients.

2. Clean Reconstruction Loss. This loss directly teaches the purifier to recover the clean image:

$$\mathcal{L}_{\text{rec}} = \|\hat{\mathbf{x}}_0 - \mathbf{x}_{\text{clean}}\|_1. \quad (21)$$

3. Label Preservation Loss. To reduce semantic drift, we optionally use a frozen clean classifier C :

$$\mathcal{L}_{\text{cls}} = \text{CE}(C(\hat{\mathbf{x}}_0), y). \quad (22)$$

4. Identity Loss. For clean identity pairs, $\delta = 0$. The model should preserve clean samples:

$$\mathcal{L}_{\text{id}} = \left\| f_\theta(\mathbf{x}_t^{\text{clean}}, t) - \mathbf{x}_{\text{clean}} \right\|_1, \quad (23)$$

where

$$\mathbf{x}_t^{\text{clean}} = \sqrt{\bar{\alpha}_t} \mathbf{x}_{\text{clean}} + \sqrt{1 - \bar{\alpha}_t} \epsilon. \quad (24)$$

The full objective is:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{distill}} + \lambda_2 \mathcal{L}_{\text{rec}} + \lambda_3 \mathcal{L}_{\text{cls}} + \lambda_4 \mathcal{L}_{\text{id}}. \quad (25)$$

3 Algorithms and Implementation Details

The proposed methodology is divided into three stages: offline poison-pair generation, poison-aware consistency training, and fast inference-time dataset sanitization.

3.1 Offline Dataset Generation

Algorithm 1 Offline Poison-Pair Dataset Generation

Require: Clean CIFAR-10 dataset $\mathcal{D}_{\text{clean}}$, Witches' Brew generator A_{WB} , Bullseye Polytope generator A_{BP} , perturbation bound ϵ , fixed random seed

Ensure: Paired purifier-training dataset $\mathcal{D}_{\text{pair}}$ and held-out evaluation cases

```
1: Initialize  $\mathcal{D}_{\text{pair}} \leftarrow \emptyset$ 
2: for each class  $c \in \{0, \dots, 9\}$  do
3:   Collect all training image indices with label  $c$ 
4:   Shuffle the class indices using the fixed random seed
5:   Allocate  $[0, 499]$  as Witches' Brew training case 1
6:   Allocate  $[500, 999]$  as Witches' Brew training case 2
7:   Allocate  $[1000, 1499]$  as Witches' Brew evaluation case
8:   Allocate  $[1500, 2499]$  as Bullseye Polytope training pool
9:   Allocate  $[2500, 2519]$  as Bullseye Polytope evaluation pool
10:  Allocate  $[2520, 3519]$  as clean identity training pool
11:  Allocate  $[3520, 4999]$  as reserve and target-selection pool
12:  for each Witches' Brew training block of 500 base images do
13:    Select a target image  $\mathbf{x}^t$  from a class different from  $c$ 
14:    Set adversarial label  $y^{\text{adv}} \leftarrow c$ 
15:    Generate poisoned images using  $A_{\text{WB}}$ 
16:    for each generated poison pair do
17:      Store  $(\mathbf{x}_{\text{clean}}, \mathbf{x}_{\text{poison}}, y, m)$  in  $\mathcal{D}_{\text{pair}}$ 
18:    end for
19:  end for
20:  Split the Bullseye Polytope training pool into 100 groups of 10 base images
21:  for each Bullseye Polytope group of 10 base images do
22:    Select a target image  $\mathbf{x}^t$  from a class different from  $c$ 
23:    Set poison label  $y^{\text{poison}} \leftarrow c$ 
24:    Generate poisoned images using  $A_{\text{BP}}$ 
25:    for each generated poison pair do
26:      Store  $(\mathbf{x}_{\text{clean}}, \mathbf{x}_{\text{poison}}, y, m)$  in  $\mathcal{D}_{\text{pair}}$ 
27:    end for
28:  end for
29:  for each clean image in the clean identity training pool do
30:    Store  $(\mathbf{x}_{\text{clean}}, \mathbf{x}_{\text{clean}}, y, m_{\text{clean}})$  in  $\mathcal{D}_{\text{pair}}$ 
31:  end for
32:  Save Witches' Brew and Bullseye Polytope evaluation blocks separately
33: end for
34: Return  $\mathcal{D}_{\text{pair}}$ 
```

The offline generation stage separates expensive poison optimization from purifier training. Once the paired dataset is saved, the CM training loop can simply load pairs and sample Gaussian noise and timesteps online.

For Bullseye Polytope, multiple independent cases can be processed in parallel using GPU batching. If B cases are batched and each case uses k poisons, then the poison tensor has shape:

$$[B, k, C, H, W]. \quad (26)$$

The batched objective is:

$$\mathcal{L}_{\text{BP}}^{\text{batch}} = \frac{1}{B} \sum_{b=1}^B \left\| \phi(\mathbf{x}_t^{(b)}) - \frac{1}{k} \sum_{j=1}^k \phi(\mathbf{x}_{p,b}^{(j)}) \right\|_2^2. \quad (27)$$

This is only GPU vectorization; each attack case remains independent.

3.2 Training Process

To avoid an overlong algorithm block, we first define the quantities used in each training step.

For a minibatch sample $(\mathbf{x}_{\text{clean}}^{(i)}, \mathbf{x}_{\text{poison}}^{(i)}, y^{(i)}, a^{(i)})$, the poison residual is

$$\delta^{(i)} = \mathbf{x}_{\text{poison}}^{(i)} - \mathbf{x}_{\text{clean}}^{(i)}. \quad (28)$$

The poison-aware noised input is

$$\mathbf{x}_t^{*,(i)} = \sqrt{\bar{\alpha}_t} \mathbf{x}_{\text{clean}}^{(i)} + \sqrt{1 - \bar{\alpha}_t} (\epsilon^{(i)} + \gamma_{a^{(i)}} \delta^{(i)}), \quad \epsilon^{(i)} \sim \mathcal{N}(0, \mathbf{I}). \quad (29)$$

The student prediction is

$$\hat{\mathbf{x}}_0^{(i)} = f_\theta(\mathbf{x}_t^{*,(i)}, t). \quad (30)$$

The total training objective is

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{distill}} + \lambda_2 \mathcal{L}_{\text{rec}} + \lambda_3 \mathcal{L}_{\text{cls}} + \lambda_4 \mathcal{L}_{\text{id}}. \quad (31)$$

Algorithm 2 Poison-Aware Consistency Distillation

Require: Offline paired dataset $\mathcal{D}_{\text{pair}}$, student purifier f_θ , EMA teacher f_{θ^-} , frozen classifier C , ODE solver Ψ , timestep schedule $\{t_n\}_{n=1}^N$

Ensure: Trained purifier P_θ

- 1: **while** not converged **do**
 - 2: Sample minibatch $\{(\mathbf{x}_{\text{clean}}^{(i)}, \mathbf{x}_{\text{poison}}^{(i)}, y^{(i)}, a^{(i)})\}_{i=1}^B$ from $\mathcal{D}_{\text{pair}}$
 - 3: Compute poison residuals $\delta^{(i)} = \mathbf{x}_{\text{poison}}^{(i)} - \mathbf{x}_{\text{clean}}^{(i)}$
 - 4: Sample timestep $t = t_{n+1}$ and Gaussian noise $\epsilon^{(i)} \sim \mathcal{N}(0, \mathbf{I})$
 - 5: Construct poison-aware noised inputs $\mathbf{x}_t^{*,(i)}$
 - 6: Predict clean images $\hat{\mathbf{x}}_0^{(i)} = f_\theta(\mathbf{x}_t^{*,(i)}, t)$
 - 7: Estimate shallower teacher inputs $\hat{\mathbf{x}}_{t_n}^{\Psi,(i)}$ using solver Ψ
 - 8: Compute $\mathcal{L}_{\text{distill}}$ using student and EMA-teacher predictions
 - 9: Compute $\mathcal{L}_{\text{rec}} = \|\hat{\mathbf{x}}_0^{(i)} - \mathbf{x}_{\text{clean}}^{(i)}\|_1$
 - 10: Compute $\mathcal{L}_{\text{cls}} = \text{CE}(C(\hat{\mathbf{x}}_0^{(i)}), y^{(i)})$
 - 11: Compute identity loss $\mathcal{L}_{\text{id}}$ on clean identity samples
 - 12: Combine losses into $\mathcal{L}_{\text{total}}$
 - 13: Update student parameters $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{total}}$
 - 14: Update EMA teacher $\theta^- \leftarrow \mu \theta^- + (1 - \mu) \theta$
 - 15: **end while**
 - 16: Return trained purifier P_θ
-

Algorithm 2 trains the proposed poison-aware CM purifier. The key step is the poison-aware noising process, where the poison residual is treated as structured noise. This allows the model to learn how to remove attack-specific perturbations while still satisfying the CM boundary condition.

- **Poison residual extraction:** The exact perturbation δ is computed from each offline pair.
- **Poison-aware noising:** The poison residual is included in the corruption distribution, so the model learns to remove both Gaussian noise and structured poison noise.
- **One-step recovery:** The model is trained to recover $\mathbf{x}_{\text{clean}}$ directly, reducing inference cost compared with iterative diffusion purification.
- **Identity preservation:** Clean identity pairs prevent the purifier from damaging benign images.

3.3 Inference Process

Algorithm 3 Inference-Time Dataset Sanitization

Require: Untrusted dataset $\mathcal{D}_{\text{untrusted}} = \{(\mathbf{x}_{\text{untrusted}}^{(i)}, y^{(i)})\}$, trained purifier f_{θ} , target timestep t^*

Ensure: Sanitized dataset $\mathcal{D}_{\text{san}}$

- 1: Initialize $\mathcal{D}_{\text{san}} \leftarrow \emptyset$
- 2: **for** each minibatch $\{(\mathbf{x}_{\text{untrusted}}^{(i)}, y^{(i)})\}_{i=1}^B$ from $\mathcal{D}_{\text{untrusted}}$ **do**
- 3: Sample standard Gaussian noise:

$$\epsilon^{(i)} \sim \mathcal{N}(0, \mathbf{I})$$

- 4: Add forward diffusion noise up to t^* :

$$\mathbf{x}_{t^*}^{(i)} = \sqrt{\bar{\alpha}_{t^*}} \mathbf{x}_{\text{untrusted}}^{(i)} + \sqrt{1 - \bar{\alpha}_{t^*}} \epsilon^{(i)}$$

- 5: Perform one-step purification:

$$\hat{\mathbf{x}}^{(i)} = f_{\theta}(\mathbf{x}_{t^*}^{(i)}, t^*)$$

- 6: Store $(\hat{\mathbf{x}}^{(i)}, y^{(i)})$ in $\mathcal{D}_{\text{san}}$
 - 7: **end for**
 - 8: Train downstream victim model on sanitized dataset $\mathcal{D}_{\text{san}}$
 - 9: Return $\mathcal{D}_{\text{san}}$
-

Algorithm 3 describes the deployment phase. The purifier is applied blindly to all training images, whether clean or poisoned. This is important because a realistic defender does not know which samples have been poisoned.

Compared with diffusion purification, the proposed CM purifier has the following advantages:

- **Faster inference:** one neural function evaluation instead of many denoising steps.
- **Dataset-level scalability:** purification can be batched on GPU, making it practical to sanitize many training images.
- **Attack diversity:** training uses both Witches’ Brew and Bullseye Polytope pairs, so the purifier is not restricted to one poison distribution.
- **Clean-image preservation:** clean identity pairs reduce unnecessary distortion of benign images.

3.4 Pre-generated Evaluation Cases from the Training Split

The evaluation cases are also generated offline, but they are kept disjoint from the purifier-training pairs. This is important because data poisoning is a train-time attack: the attacker poisons training images, not the official CIFAR-10 test images. Therefore, our held-out poisoning evaluation cases are pre-generated from unused CIFAR-10 training images.

For each class c , the manifest allocator reserves separate blocks for training and evaluation (mentioned in section 2.3). The blocks [1000, 1499] (for Witches’ Brew) and [2500, 2519] (Bullseye Polytope) are not used to train the purifier. They are used only to generate held-out poisoned training sets for final defense evaluation. This gives us a controlled evaluation setting where the purifier is tested on poison cases that were not seen during CM training.

For Witches’ Brew evaluation, each class contributes one held-out case with 500 poisoned base images. The victim model is trained from scratch on the resulting poisoned training set. We compare:

- training on the clean dataset;
- training on the poisoned dataset;
- training on the purified poisoned dataset.

The same victim initialization seed and training schedule are used across all three conditions.

For Bullseye Polytope evaluation, each class contributes held-out 2 groups of 10 base images. These held-out groups are used to generate fresh Bullseye Polytope poisons. The primary evaluation uses the transfer-learning setting, since Bullseye Polytope is designed around feature-space poisoning. Again, we compare clean, poisoned, and purified-poisoned training under the same victim seed.

The official CIFAR-10 test set is kept separate and is used to measure clean test accuracy and target misclassification after the downstream victim model is trained. Thus, the evaluation protocol uses:

- held-out CIFAR-10 training images for pre-generated poison evaluation cases;
- the official CIFAR-10 test set for final classification accuracy and attack-success measurement.

3.5 Metrics

- attack success rate before and after purification;

- clean test accuracy;
- target confidence for the adversarial label;
- average purification time per image;
- total dataset sanitization time.

References

- [1] Jonas Geiping, Liam Fowl, W. Ronny Huang, Wojciech Czaja, Gavin Taylor, Michael Moeller, and Tom Goldstein. Witches’ brew: Industrial scale data poisoning via gradient matching, 2021.
- [2] Hojjat Aghakhani, Dongyu Meng, Yu-Xiang Wang, Christopher Kruegel, and Giovanni Vigna. Bullseye polytope: A scalable clean-label poisoning attack with improved transferability. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2021.
- [3] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning (ICML)*, pages 32211–32252. PMLR, 2023.
- [4] Chun Tong Lei, Hon Ming Yam, Zhongliang Guo, Yifei Qian, and Chun Pong Lau. Instant adversarial purification with adversarial consistency distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.